

NumPy for MATLAB users

Introduction

MATLAB® and NumPy have a lot in common, but NumPy was created to work with Python, not to be a MATLAB clone. This guide will help MATLAB users get started with NumPy.

Some key differences

In MATLAB, the basic type, even for scalars, is a multidimensional array. Array assignments in MATLAB are stored as 2D arrays of double precision floating point numbers, unless you specify the number of dimensions and type. Operations on the 2D instances of these arrays are modeled on matrix operations in linear algebra.	In NumPy, the basic type is a multidimensional <code>array</code> . Array assignments in NumPy are usually stored as n-dimensional arrays with the minimum type required to hold the objects in sequence, unless you specify the number of dimensions and type. NumPy performs operations element-by-element, so multiplying 2D arrays with <code>*</code> is not a matrix multiplication – it's an element-by-element multiplication. (The <code>@</code> operator, available since Python 3.5, can be used for conventional matrix multiplication.)
MATLAB numbers indices from 1; <code>a(1)</code> is the first element. See note INDEXING	NumPy, like Python, numbers indices from 0; <code>a[0]</code> is the first element.
MATLAB's scripting language was created for linear algebra so the syntax for some array manipulations is more compact than NumPy's. On the other hand, the API for adding GUIs and creating full-fledged applications is more or less an afterthought.	NumPy is based on Python, a general-purpose language. The advantage to NumPy is access to Python libraries including: SciPy , Matplotlib , Pandas , OpenCV , and more. In addition, Python is often embedded as a scripting language in other software, allowing NumPy to be used there too.
MATLAB array slicing uses pass-by-value semantics, with a lazy copy-on-write scheme to prevent creating copies until they are needed. Slicing operations copy parts of the array.	NumPy array slicing uses pass-by-reference, that does not copy the arguments. Slicing operations are views into an array.

Rough equivalents

The table below gives rough equivalents for some common MATLAB expressions. These are similar expressions, not equivalents. For details, see the [documentation](#).

In the table below, it is assumed that you have executed the following commands in Python:

```
import numpy as np
from scipy import io, integrate, linalg, signal
from scipy.sparse.linalg import cg, eigs
```

Also assume below that if the Notes talk about “matrix” that the arguments are two-dimensional entities.

General purpose equivalents

MATLAB	NumPy	Notes
<code>help func</code>	<code>info(func)</code> or <code>help(func)</code> or <code>func?</code> (in IPython)	get help on the function <i>func</i>
<code>which func</code>	see note HELP	find out where <i>func</i> is defined
<code>type func</code>	<code>np.source(func)</code> or <code>func??</code> (in IPython)	print source for <i>func</i> (if not a native function)
<code>% comment</code>	<code># comment</code>	comment a line of code with the text <code>comment</code>
<pre>for i=1:3 fprintf f('%i\ n',i) end</pre>	<pre>for i in range(1, 4): print(i)</pre>	use a for-loop to print the numbers 1, 2, and 3 using range
<code>a && b</code>	<code>a and b</code>	short-circuiting logical AND operator (Python native operator); scalar arguments only

MATLAB	NumPy	Notes
<code>a b</code>	<code>a or b</code>	short-circuiting logical OR operator (Python native operator); scalar arguments only
<pre>>> 4 == 4 ans = 1 >> 4 == 5 ans = 0</pre>	<pre>>>> 4 == 4 True >>> 4 == 5 False</pre>	The boolean objects in Python are <code>True</code> and <code>False</code> , as opposed to MATLAB logical types of <code>1</code> and <code>0</code> .
<pre>a=4 if a==4 fprintf f('a = 4\n') elseif a==5 fprintf f('a = 5\n') end</pre>	<pre>a = 4 if a == 4: print('a = 4') elif a == 5: print('a = 5')</pre>	create an if-else statement to check if <code>a</code> is 4 or 5 and print result
<code>1*i</code> , <code>1*j</code> , <code>1i</code> , <code>1j</code>	<code>1j</code>	complex numbers
<code>eps</code>	<code>np.finfo(float).eps</code> or <code>np.spacing(1)</code>	distance from 1 to the next larger representable real number in double precision
<code>load</code> <code>data.mat</code>	<code>io.loadmat('data.mat')</code>	Load MATLAB variables saved to the file <code>data.mat</code> . (Note: When saving arrays to <code>data.mat</code> in MATLAB/Octave, use a recent binary format. scipy.io.loadmat will create a dictionary with the saved arrays and further information.)
<code>ode45</code>	<code>integrate.solve_ivp(f)</code>	integrate an ODE with Runge-Kutta 4,5

MATLAB	NumPy	Notes
<code>ode15s</code>	<code>integrate.solve_ivp(f, method='BDF')</code>	integrate an ODE with BDF method

Linear algebra equivalents

MATLAB	NumPy	Notes
<code>ndims(a)</code>	<code>np.ndim(a)</code> or <code>a.ndim</code>	number of dimensions of array <code>a</code>
<code>numel(a)</code>	<code>np.size(a)</code> or <code>a.size</code>	number of elements of array <code>a</code>
<code>size(a)</code>	<code>np.shape(a)</code> or <code>a.shape</code>	“size” of array <code>a</code>
<code>size(a,n)</code>	<code>a.shape[n-1]</code>	get the number of elements of the n-th dimension of array <code>a</code> . (Note that MATLAB uses 1 based indexing while Python uses 0 based indexing, See note INDEXING)
<code>[1 2 3; 4 5 6]</code>	<code>np.array([[1., 2., 3.], [4., 5., 6.]])</code>	define a 2x3 2D array
<code>[a b; c d]</code>	<code>np.block([[a, b], [c, d]])</code>	construct a matrix from blocks <code>a</code> , <code>b</code> , <code>c</code> , and <code>d</code>

MATLAB	NumPy	Notes
<code>a(end)</code>	<code>a[-1]</code>	access last element in MATLAB vector (1xn or nx1) or 1D NumPy array <code>a</code> (length n)
<code>a(2,5)</code>	<code>a[1, 4]</code>	access element in second row, fifth column in 2D array <code>a</code>
<code>a(2,:)</code>	<code>a[1]</code> or <code>a[1, :]</code>	entire second row of 2D array <code>a</code>
<code>a(1:5,:)</code>	<code>a[0:5]</code> or <code>a[:5]</code> or <code>a[0:5, :]</code>	first 5 rows of 2D array <code>a</code>
<code>a(end-4:end,:)</code>	<code>a[-5:]</code>	last 5 rows of 2D array <code>a</code>
<code>a(1:3,5:9)</code>	<code>a[0:3, 4:9]</code>	The first through third rows and fifth through ninth columns of a 2D array, <code>a</code> .
<code>a([2,4,5],[1,3])</code>	<code>a[np.ix_([1, 3, 4], [0, 2])]</code>	rows 2,4 and 5 and columns 1 and 3. This allows the matrix to be modified, and doesn't require a regular slice.
<code>a(3:2:21,:)</code>	<code>a[2:21:2,:]</code>	every other row of <code>a</code> , starting

MATLAB	NumPy	Notes
		with the third and going to the twenty-first
<code>a(1:2:end,:)</code>	<code>a[::2, :]</code>	every other row of <code>a</code> , starting with the first
<code>a(end:-1:1,:)</code> or <code>flipud(a)</code>	<code>a[::-1,:]</code>	<code>a</code> with rows in reverse order
<code>a([1:end 1],:)</code>	<code>a[np.r_[0:len(a),0]]</code>	<code>a</code> with copy of the first row appended to the end
<code>a.'</code>	<code>a.transpose()</code> or <code>a.T</code>	transpose of <code>a</code>
<code>a'</code>	<code>a.conj().transpose()</code> or <code>a.conj().T</code>	conjugate transpose of <code>a</code>
<code>a * b</code>	<code>a @ b</code>	matrix multiply
<code>a .* b</code>	<code>a * b</code>	element-wise multiply
<code>a ./ b</code>	<code>a / b</code>	element-wise divide
<code>a.^3</code>	<code>a**3</code>	element-wise exponentiation
<code>(a > 0.5)</code>	<code>(a > 0.5)</code>	matrix whose i,jth element is (a _{ij} > 0.5). The MATLAB result is an array of logical values 0 and 1. The NumPy result is

MATLAB	NumPy	Notes
		an array of the boolean values <code>False</code> and <code>True</code> .
<code>find(a > 0.5)</code>	<code>np.nonzero(a > 0.5)</code>	find the indices where (<code>a</code> > 0.5)
<code>a(:,find(v > 0.5))</code>	<code>a[:,np.nonzero(v > 0.5)[0]]</code>	extract the columns of <code>a</code> where vector <code>v</code> > 0.5
<code>a(:,find(v>0.5))</code>	<code>a[:, v.T > 0.5]</code>	extract the columns of <code>a</code> where column vector <code>v</code> > 0.5
<code>a(a<0.5)=0</code>	<code>a[a < 0.5]=0</code>	<code>a</code> with elements less than 0.5 zeroed out
<code>a .* (a>0.5)</code>	<code>a * (a > 0.5)</code>	<code>a</code> with elements less than 0.5 zeroed out
<code>a(:) = 3</code>	<code>a[:] = 3</code>	set all values to the same scalar value
<code>y=x</code>	<code>y = x.copy()</code>	NumPy assigns by reference
<code>y=x(2,:) </code>	<code>y = x[1, :].copy()</code>	NumPy slices are by reference
<code>y=x(:)</code>	<code>y = x.flatten()</code>	turn array into vector (note that this forces a copy). To obtain

MATLAB	NumPy	Notes
		the same data ordering as in MATLAB, use <code>x.flatten('F')</code> .
<code>1:10</code>	<code>np.arange(1., 11.)</code> or <code>np.r_[1.:11.]</code> or <code>np.r_[1:10:10j]</code>	create an increasing vector (see note RANGES)
<code>0:9</code>	<code>np.arange(10.)</code> or <code>np.r_[:10.]</code> or <code>np.r_[:9:10j]</code>	create an increasing vector (see note RANGES)
<code>[1:10]'</code>	<code>np.arange(1.,11.)[:, np.newaxis]</code>	create a column vector
<code>zeros(3,4)</code>	<code>np.zeros((3, 4))</code>	3x4 two-dimensional array full of 64-bit floating point zeros
<code>zeros(3,4,5)</code>	<code>np.zeros((3, 4, 5))</code>	3x4x5 three-dimensional array full of 64-bit floating point zeros
<code>ones(3,4)</code>	<code>np.ones((3, 4))</code>	3x4 two-dimensional array full of 64-bit floating point ones
<code>eye(3)</code>	<code>np.eye(3)</code>	3x3 identity matrix

MATLAB	NumPy	Notes
<code>diag(a)</code>	<code>np.diag(a)</code>	returns a vector of the diagonal elements of 2D array, <code>a</code>
<code>diag(v,0)</code>	<code>np.diag(v, 0)</code>	returns a square diagonal matrix whose nonzero values are the elements of vector, <code>v</code>
<pre>rng(42,'twister') rand(3,4)</pre>	<pre>from numpy.random import default_rng rng = default_rng(42) rng.random((3, 4))</pre> or older version: <code>random.rand((3, 4))</code>	generate a random 3x4 array with default random number generator and seed = 42
<code>linspace(1,3,4)</code>	<code>np.linspace(1,3,4)</code>	4 equally spaced samples between 1 and 3, inclusive
<code>[x,y]=meshgrid(0:8,0:5)</code>	<code>np.mgrid[0:9.,0:6.]</code> or <code>np.meshgrid(r_[0:9.],r_[0:6.])</code>	two 2D arrays: one of x values, the other of y values
	<code>ogrid[0:9.,0:6.]</code> or <code>np.ix_(np.r_[0:9.],np.r_[0:6.])</code>	the best way to eval functions on a grid
<code>[x,y]=meshgrid([1,2,4],[2,4,5])</code>	<code>np.meshgrid([1,2,4],[2,4,5])</code>	
	<code>np.ix_([1,2,4],[2,4,5])</code>	the best way to eval functions on a grid

MATLAB	NumPy	Notes
<code>repmat(a, m, n)</code>	<code>np.tile(a, (m, n))</code>	create m by n copies of <code>a</code>
<code>[a b]</code>	<code>np.concatenate((a,b),1)</code> or <code>np.hstack((a,b))</code> or <code>np.column_stack((a,b))</code> or <code>np.c_[a,b]</code>	concatenate columns of <code>a</code> and <code>b</code>
<code>[a; b]</code>	<code>np.concatenate((a,b))</code> or <code>np.vstack((a,b))</code> or <code>np.r_[a,b]</code>	concatenate rows of <code>a</code> and <code>b</code>
<code>max(max(a))</code>	<code>a.max()</code> or <code>np.nanmax(a)</code>	maximum element of <code>a</code> (with <code>ndims(a) <= 2</code> for MATLAB, if there are NaN's, <code>nanmax</code> will ignore these and return largest value)
<code>max(a)</code>	<code>a.max(0)</code>	maximum element of each column of array <code>a</code>
<code>max(a,[],2)</code>	<code>a.max(1)</code>	maximum element of each row of array <code>a</code>
<code>max(a,b)</code>	<code>np.maximum(a, b)</code>	compares <code>a</code> and <code>b</code> element-wise, and returns the maximum value from each pair

MATLAB	NumPy	Notes
<code>norm(v)</code>	<code>np.sqrt(v @ v)</code> or <code>np.linalg.norm(v)</code>	L2 norm of vector <code>v</code>
<code>a & b</code>	<code>logical_and(a,b)</code>	element-by-element AND operator (NumPy ufunc) See note LOGICOPS
<code>a b</code>	<code>np.logical_or(a,b)</code>	element-by-element OR operator (NumPy ufunc) See note LOGICOPS
<code>bitand(a,b)</code>	<code>a & b</code>	bitwise AND operator (Python native and NumPy ufunc)
<code>bitor(a,b)</code>	<code>a b</code>	bitwise OR operator (Python native and NumPy ufunc)
<code>inv(a)</code>	<code>linalg.inv(a)</code>	inverse of square 2D array <code>a</code>
<code>pinv(a)</code>	<code>linalg.pinv(a)</code>	pseudo-inverse of 2D array <code>a</code>
<code>rank(a)</code>	<code>np.linalg.matrix_rank(a)</code>	matrix rank of a 2D array <code>a</code>
<code>a\b</code>	<code>linalg.solve(a, b)</code> if <code>a</code> is square; <code>linalg.lstsq(a, b)</code>	solution of $a x = b$ for x

MATLAB	NumPy	Notes
	otherwise	
<code>b/a</code>	Solve <code>a.T x.T = b.T</code> instead	solution of $x a = b$ for x
<code>[U,S,V]=svd(a)</code>	<code>U, S, Vh = linalg.svd(a); V = Vh.T</code>	singular value decomposition of <code>a</code>
<code>chol(a)</code>	<code>linalg.cholesky(a)</code>	Cholesky factorization of a 2D array
<code>[V,D]=eig(a)</code>	<code>D,V = linalg.eig(a)</code>	eigenvalues λ and eigenvectors v of <code>a</code> , where $av = \lambda v$
<code>[V,D]=eig(a,b)</code>	<code>D,V = linalg.eig(a, b)</code>	eigenvalues λ and eigenvectors v of <code>a</code> , <code>b</code> where $av = \lambda bv$
<code>[V,D]=eigs(a,3)</code>	<code>D,V = eigs(a, k=3)</code>	find the <code>k=3</code> largest eigenvalues and eigenvectors of 2D array, <code>a</code>
<code>[Q,R]=qr(a,0)</code>	<code>Q,R = linalg.qr(a)</code>	QR decomposition

MATLAB	NumPy	Notes
<code>[L,U,P]=lu(a)</code> where <code>a==P'*L*U</code>	<code>P,L,U = linalg.lu(a)</code> where <code>a == P@L@U</code>	LU decomposition with partial pivoting (note: <code>P(MATLAB) == transpose(P(NumPy))</code>)
<code>conjgrad</code>	<code>cg</code>	conjugate gradients solver
<code>fft(a)</code>	<code>np.fft.fft(a)</code>	Fourier transform of <code>a</code>
<code>ifft(a)</code>	<code>np.fft.ifft(a)</code>	inverse Fourier transform of <code>a</code>
<code>sort(a)</code>	<code>np.sort(a)</code> or <code>a.sort(axis=0)</code>	sort each column of a 2D array, <code>a</code>
<code>sort(a, 2)</code>	<code>np.sort(a, axis=1)</code> or <code>a.sort(axis=1)</code>	sort the each row of 2D array, <code>a</code>
<code>[b,I]=sortrows(a,1)</code>	<code>I = np.argsort(a[:, 0]); b = a[I,:]</code>	save the array <code>a</code> as array <code>b</code> with rows sorted by the first column
<code>x = Z\y</code>	<code>x = linalg.lstsq(Z, y)</code>	perform a linear regression of the form Zx = y
<code>decimate(x, q)</code>	<code>signal.resample(x, np.ceil(len(x)/q))</code>	downsample with low-pass filtering
<code>unique(a)</code>	<code>np.unique(a)</code>	a vector of unique values in

MATLAB	NumPy	Notes
		array <code>a</code>
<code>squeeze(a)</code>	<code>a.squeeze()</code>	remove singleton dimensions of array <code>a</code> . Note that MATLAB will always return arrays of 2D or higher while NumPy will return arrays of 0D or higher

Notes

Submatrix: Assignment to a submatrix can be done with lists of indices using the `ix_` command. E.g., for 2D array `a`, one might do: `ind=[1, 3]; a[np.ix_(ind, ind)] += 100`.

HELP: There is no direct equivalent of MATLAB's `which` command, but the commands `help` will usually list the filename where the function is located. Python also has an `inspect` module (do `import inspect`) which provides a `getfile` that often works.

INDEXING: MATLAB uses one based indexing, so the initial element of a sequence has index 1. Python uses zero based indexing, so the initial element of a sequence has index 0. Confusion and flamewars arise because each has advantages and disadvantages. One based indexing is consistent with common human language usage, where the “first” element of a sequence has index 1. Zero based indexing [simplifies indexing](#). See also [a text by prof.dr. Edsger W. Dijkstra](#).

RANGES: In MATLAB, `0:5` can be used as both a range literal and a ‘slice’ index (inside parentheses); however, in Python, constructs like `0:5` can *only* be used as a slice index (inside square brackets). Thus the somewhat quirky `r_` object was created to allow NumPy to have a similarly terse range construction mechanism. Note that `r_` is not called like a function or a constructor, but rather *indexed* using square brackets, which allows the use of Python’s slice syntax in the arguments.

LOGICOPS: `&` or `|` in NumPy is bitwise AND/OR, while in MATLAB `&` and `|` are logical AND/OR. The two can appear to work the same, but there are important differences. If you would have used MATLAB's `&` or `|` operators, you should use the NumPy ufuncs `logical_and` / `logical_or`. The notable differences between MATLAB's `&` and `|` operators are:

- Non-logical {0,1} inputs: NumPy's output is the bitwise AND of the inputs. MATLAB treats any non-zero value as 1 and returns the logical AND. For example `(3 & 4)` in NumPy is `0`, while in MATLAB both `3` and `4` are considered logical true and `(3 & 4)` returns `1`.
- Precedence: NumPy's `&` operator is higher precedence than logical operators like `<` and `>`; MATLAB's is the reverse.

If you know you have boolean arguments, you can get away with using NumPy's bitwise operators, but be careful with parentheses, like this: `z`

`= (x > 1) & (x < 2)`. The absence of NumPy operator forms of `logical_and` and `logical_or` is an unfortunate consequence of Python's design.

RESHAPE and LINEAR INDEXING: MATLAB always allows multi-dimensional arrays to be accessed using scalar or linear indices, NumPy does not. Linear indices are common in MATLAB programs, e.g. `find()` on a matrix returns them, whereas NumPy's `find` behaves differently. When converting MATLAB code it might be necessary to first reshape a matrix to a linear sequence, perform some indexing operations and then reshape back. As reshape (usually) produces views onto the same storage, it should be possible to do this fairly efficiently. Note that the scan order used by reshape in NumPy defaults to the 'C' order, whereas MATLAB uses the Fortran order. If you are simply converting to a linear sequence and back this doesn't matter. But if you are converting reshapes from MATLAB code which relies on the scan order, then this MATLAB code: `z = reshape(x,3,4);` should become `z = x.reshape(3,4,order='F').copy()` in NumPy.

'array' or 'matrix'? Which should I use?

Historically, NumPy has provided a special matrix type, `np.matrix`, which is a subclass of `ndarray` which makes binary operations linear algebra operations. You may see it used in some existing code instead of `np.array`. So, which one to use?

Short answer

Use arrays.

- They support multidimensional array algebra that is supported in MATLAB
- They are the standard vector/matrix/tensor type of NumPy. Many NumPy functions return arrays, not matrices.
- There is a clear distinction between element-wise operations and linear algebra operations.
- You can have standard vectors or row/column vectors if you like.

Until Python 3.5 the only disadvantage of using the array type was that you had to use `dot` instead of `*` to multiply (reduce) two tensors (scalar product, matrix vector multiplication etc.). Since Python 3.5 you can use the matrix multiplication `@` operator.

Given the above, we intend to deprecate `matrix` eventually.

Long answer

NumPy contains both an `array` class and a `matrix` class. The `array` class is intended to be a general-purpose n-dimensional array for many kinds of numerical computing, while `matrix` is intended to facilitate linear algebra computations specifically. In practice there are only a handful of key differences between the two.

- Operators `*` and `@`, functions `dot()`, and `multiply()`:
 - For `array`, `*` means **element-wise multiplication**, while `@` means **matrix multiplication**; they have associated functions `multiply()` and `dot()`.
 - For `matrix`, `*` means **matrix multiplication**, and for element-wise multiplication one has to use the `multiply()` function.
- Handling of vectors (one-dimensional arrays)
 - For `array`, the **vector shapes 1xN, Nx1, and N are all different things**. Operations like `A[:,1]` return a one-dimensional array of shape N, not a two-dimensional array of shape Nx1. Transpose on a one-dimensional `array` does nothing.
 - For `matrix`, **one-dimensional arrays are always upconverted to 1xN or Nx1 matrices** (row or column vectors). `A[:,1]` returns a two-dimensional matrix of shape Nx1.
- Handling of higher-dimensional arrays (ndim > 2)
 - `array` objects **can have number of dimensions > 2**;
 - `matrix` objects **always have exactly two dimensions**.
- Convenience attributes
 - `array` has a `.T` attribute, which returns the transpose of the data.

- `matrix` also has `.H`, `.I`, and `.A` attributes, which return the conjugate transpose, inverse, and `asarray()` of the matrix, respectively.
- Convenience constructor
 - The `array` constructor takes (nested) Python sequences as initializers. As in, `array([[1,2,3],[4,5,6]])`.
 - The `matrix` constructor additionally takes a convenient string initializer. As in `matrix("[1 2 3; 4 5 6]")`.

There are pros and cons to using both:

- `array`
 - `:` Element-wise multiplication is easy: `A*B`.
 - `:` You have to remember that matrix multiplication has its own operator, `@`.
 - `:` You can treat one-dimensional arrays as *either* row or column vectors. `A @ v` treats `v` as a column vector, while `v @ A` treats `v` as a row vector. This can save you having to type a lot of transposes.
 - `:` `array` is the “default” NumPy type, so it gets the most testing, and is the type most likely to be returned by 3rd party code that uses NumPy.
 - `:` Is quite at home handling data of any number of dimensions.
 - `:` Closer in semantics to tensor algebra, if you are familiar with that.
 - `:` All operations (`*`, `/`, `+`, `-` etc.) are element-wise.
 - `:` Sparse matrices from `scipy.sparse` do not interact as well with arrays.
- `matrix`
 - `:` Behavior is more like that of MATLAB matrices.
 - `<:` Maximum of two-dimensional. To hold three-dimensional data you need `array` or perhaps a Python list of `matrix`.
 - `<:` Minimum of two-dimensional. You cannot have vectors. They must be cast as single-column or single-row matrices.
 - `<:` Since `array` is the default in NumPy, some functions may return an `array` even if you give them a `matrix` as an argument. This shouldn't happen with NumPy functions (if it does it's a bug), but 3rd party code based on NumPy may not honor type preservation like NumPy does.
 - `:` `A*B` is matrix multiplication, so it looks just like you write it in linear algebra (For Python $\geq$ 3.5 plain arrays have the same convenience with the `@` operator).
 - `<:` Element-wise multiplication requires calling a function, `multiply(A,B)`.
 - `<:` The use of operator overloading is a bit illogical: `*` does not work element-wise but `/` does.

- Interaction with `scipy.sparse` is a bit cleaner.

The `array` is thus much more advisable to use. Indeed, we intend to deprecate `matrix` eventually.

Customizing your environment

In MATLAB the main tool available to you for customizing the environment is to modify the search path with the locations of your favorite functions. You can put such customizations into a startup script that MATLAB will run on startup.

NumPy, or rather Python, has similar facilities.

- To modify your Python search path to include the locations of your own modules, define the `PYTHONPATH` environment variable.
- To have a particular script file executed when the interactive Python interpreter is started, define the `PYTHONSTARTUP` environment variable to contain the name of your startup script.

Unlike MATLAB, where anything on your path can be called immediately, with Python you need to first do an 'import' statement to make functions in a particular file accessible.

For example you might make a startup script that looks like this (Note: this is just an example, not a statement of "best practices"):

```
# Make all numpy available via shorter 'np' prefix
import numpy as np
#
# Make the SciPy linear algebra functions available as linalg.func()
# e.g. linalg.lu, linalg.eig (for general  $l^*B@u==A@u$  solution)
from scipy import linalg
#
# Define a Hermitian function
def hermitian(A, **kwargs):
    return np.conj(A, **kwargs).T
# Make a shortcut for hermitian:
#     hermitian(A) --> H(A)
H = hermitian
```

To use the deprecated *matrix* and other *matlib* functions:

```
# Make all matlib functions accessible at the top level via M.func()  
import numpy.matlib as M  
# Make some matlib functions accessible directly at the top level via, e.g.  
rand(3,3)  
from numpy.matlib import matrix,rand,zeros,ones,empty,eye
```

Links

Another somewhat outdated MATLAB/NumPy cross-reference can be found at

<https://mathesaurus.sf.net/>

An extensive list of tools for scientific work with Python can be found in the [topical software page](#).

See [List of Python software: scripting](#) for a list of software that use Python as a scripting language

MATLAB® and SimuLink® are registered trademarks of The MathWorks, Inc.